

Date:

Exp No:3

Aim:Implement process scheduling algorithms(FCFS,SJF,RoundRobin,Priority) and compute average waiting time and average turn around time.

1.FCFS(First Come First Serve)

Theory

First Come First Serve (FCFS) is the simplest CPU scheduling algorithm.

In this algorithm, the process that arrives first in the ready queue is executed first, just like a queue system.

FCFS follows non-preemptive scheduling, which means:

- Once a process starts execution, it runs until completion
- CPU cannot be taken away by another process

Working Principle

1. Processes enter the ready queue based on their arrival time
2. CPU selects the process at the front of the queue
3. After completion, the next process is executed

Scheduling Criteria

Waiting Time (WT)

The time a process waits in the ready queue before execution.

$WT = \text{Turnaround Time} - \text{Burst Time}$

Turnaround Time (TAT)

The total time taken from process arrival to completion.

$TAT = \text{Completion Time} - \text{Arrival Time}$

1.Write the c program OS_exp3.c

```
#include <stdio.h>
```

```
int main() {  
    int n, i;  
    int at[20], bt[20], wt[20], tat[20], ct[20];  
    float avg_wt = 0, avg_tat = 0;
```

```

printf("Enter number of processes: ");
scanf("%d", &n);

for (i = 0; i < n; i++) {
    printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
    scanf("%d %d", &at[i], &bt[i]);
}

ct[0] = at[0] + bt[0];

for (i = 1; i < n; i++) {
    if (ct[i - 1] < at[i])
        ct[i] = at[i] + bt[i]; // CPU idle case
    else
        ct[i] = ct[i - 1] + bt[i];
}

for (i = 0; i < n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("\nProcess\tAT\tBT\tWT\tTAT\n");
for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time = %.2f", avg_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

```

```
    return 0;
}
```

2.Compile and Run

```
gcc OS_exp3.c
./a.out
```

3.Expected Output

```
user@user-HP-Laptop-15s-eq2xxx:~/Desktop$ ./a.out
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 2
Enter Arrival Time and Burst Time for P2: 1 3
Enter Arrival Time and Burst Time for P3: 2 4

Process AT      BT      WT      TAT
P1      0        2        0        2
P2      1        3        1        4
P3      2        4        3        7

Average Waiting Time = 1.33
Average Turnaround Time = 4.33
```

ii)SJF(Shortest Job First)

Theory

Shortest Job First (SJF) is a CPU scheduling algorithm in which the process having the smallest burst time is selected for execution next.

Working Principle

1. All processes enter the ready queue based on their arrival time.
2. Among the available processes, the one with the shortest burst time is selected.
3. The selected process executes completely (non-preemptive).
4. After completion, the next shortest job is chosen.
5. If no process has arrived, the CPU remains idle.

Scheduling Criteria

Completion Time (CT)

The time at which a process completes execution.

Turnaround Time (TAT)

The total time taken by a process from arrival to completion.

$$TAT = CT - AT$$

Waiting Time (WT)

The time a process waits in the ready queue.

$$WT = TAT - BT$$

1. Write the c program OS_exp3_1.c

```
#include <stdio.h>
```

```
int main() {
    int n, i, time = 0, completed = 0;
    int at[20], bt[20], ct[20], wt[20], tat[20];
    int done[20] = {0};
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    while (completed < n) {
        int idx = -1;
        int min_bt = 9999;

        // find shortest job among arrived processes
        for (i = 0; i < n; i++) {
            if (at[i] <= time && done[i] == 0 && bt[i] < min_bt) {
                min_bt = bt[i];
                idx = i;
            }
        }

        if (idx == -1) {
            time++; // CPU idle
        } else {
            time += bt[idx];
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];

            avg_wt += wt[idx];
        }

        completed++;
    }

    printf("Average Waiting Time: %.2f\n", avg_wt / n);
    printf("Average Turnaround Time: %.2f\n", avg_tat / n);
}
```

```

        avg_tat += tat[idx];

        done[idx] = 1;
        completed++;
    }
}

printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], ct[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time = %.2f", avg_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

return 0;
}

```

2.Compile and Run

```

gcc OS_exp3_1.c
./a.out

```

3.Expected Output

```

Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 3
Enter Arrival Time and Burst Time for P2: 1 2
Enter Arrival Time and Burst Time for P3: 2 4

Process AT      BT      CT      WT      TAT
P1      0      3      3      0      3
P2      1      2      5      2      4
P3      2      4      9      3      7

Average Waiting Time = 1.67
Average Turnaround Time = 4.67

```

iii)Round Robin(Non-preemptive)

Theory

Round Robin (RR) is a preemptive CPU scheduling algorithm designed mainly for time-sharing systems.

In this algorithm, each process is assigned a fixed time slot called Time Quantum (TQ).

The CPU executes a process for one time quantum.

- If the process finishes within the time quantum, it releases the CPU.
- If not, it is preempted and placed at the end of the ready queue.

Working Principle

1. All processes enter the ready queue based on their arrival time.
2. The CPU selects the first process in the queue.
3. The process runs for minimum of (Burst Time, Time Quantum).
4. If the process is not completed, it is moved to the end of the queue.
5. The CPU then schedules the next process.
6. This continues until all processes are completed.
7. If no process is available, the CPU remains idle.

Scheduling Criteria

Completion Time (CT)

The time at which a process completes execution.

Turnaround Time (TAT)

The total time taken by a process from arrival to completion.

$$TAT = CT - AT$$

Waiting Time (WT)

The time a process waits in the ready queue.

$$WT = TAT - BT$$

1. Write the c program OS_exp3_2.c

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, tq;
```

```
    int at[20], bt[20], rt[20];
```

```
    int ct[20], wt[20], tat[20];
```

```
    int time = 0, completed = 0;
```

```

int i, found;

float avg_wt = 0, avg_tat = 0;


printf("Enter number of processes: ");
scanf("%d", &n);


for (i = 0; i < n; i++) {
    printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
    scanf("%d %d", &at[i], &bt[i]);
    rt[i] = bt[i]; // remaining time
}


printf("Enter Time Quantum: ");
scanf("%d", &tq);


while (completed < n) {
    found = 0;

    for (i = 0; i < n; i++) {
        if (at[i] <= time && rt[i] > 0) {
            found = 1;

            if (rt[i] > tq) {
                time += tq;
                rt[i] -= tq;
            } else {
                time += rt[i];
                ct[i] = time;
                rt[i] = 0;
                completed++;
            }
        }
    }
}

```

```

    }

    if (!found)
        time++; // CPU idle
    }

    for (i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            i + 1, at[i], bt[i], ct[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avg_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

    return 0;
}

```

2.Compile and Run

```

gcc OS_exp3_2.c
./a.out

```


3.Expected output

```
Enter number of processes: 5
Enter Arrival Time and Burst Time for P1: 0 5
Enter Arrival Time and Burst Time for P2: 1 4
Enter Arrival Time and Burst Time for P3: 3 7
Enter Arrival Time and Burst Time for P4: 2 3
Enter Arrival Time and Burst Time for P5: 4 1
Enter Time Quantum: 3

Process AT      BT      CT      WT      TAT
P1      0       5      15      10      15
P2      1       4      16      11      15
P3      3       7      20      10      17
P4      2       3      12       7      10
P5      4       1      13       8       9

Average Waiting Time = 9.20
Average Turnaround Time = 13.20
```

iv)Priority Scheduling

Theory

Priority Scheduling is a CPU scheduling algorithm in which each process is assigned a priority, and the CPU is allocated to the process with the highest priority.

Working Principle

1. All processes enter the ready queue based on their arrival time.
2. Among the arrived processes, the one with the highest priority is selected.
3. The selected process executes on the CPU.
4. After completion (or preemption), the next highest-priority process is chosen.
5. If no process is available, the CPU remains idle.

Scheduling Criteria

Completion Time (CT)

The time at which a process completes execution.

Turnaround Time (TAT)

The total time taken by a process from arrival to completion.

$$TAT = CT - AT$$

Waiting Time (WT)

The time a process waits in the ready queue.

$$WT = TAT - BT$$

1. Write the c program OS_exp3_2.c

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, time = 0, completed = 0;
```

```
    int at[20], bt[20], pr[20];
```

```
    int ct[20], wt[20], tat[20];
```

```
    int done[20] = {0};
```

```
    float avg_wt = 0, avg_tat = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("Enter Arrival Time, Burst Time and Priority for P%d: ", i + 1);
```

```
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
```

```
    }
```

```
    while (completed < n) {
```

```
        int idx = -1;
```

```
        int highest_priority = 9999;
```

```
        // find highest priority process among arrived processes
```

```
        for (i = 0; i < n; i++) {
```

```
            if (at[i] <= time && done[i] == 0 && pr[i] < highest_priority) {
```

```
                highest_priority = pr[i];
```

```
                idx = i;
```

```
            }
```

```
        }
```

```
        if (idx == -1) {
```

```

        time++; // CPU idle
    } else {
        time += bt[idx];
        ct[idx] = time;
        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];

        avg_wt += wt[idx];
        avg_tat += tat[idx];

        done[idx] = 1;
        completed++;
    }
}

printf("\nProcess\tAT\tBT\tPR\tCT\tWT\tTAT\n");
for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
           i + 1, at[i], bt[i], pr[i], ct[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time = %.2f", avg_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

return 0;
}

```

2.Compile and Run

```

gcc OS_exp3_3.c
./a.out

```

3.Expected Output

```
Enter number of processes: 4
Enter Arrival Time, Burst Time and Priority for P1: 0 4 0
Enter Arrival Time, Burst Time and Priority for P2: 1 2 1
Enter Arrival Time, Burst Time and Priority for P3: 3 5 2
Enter Arrival Time, Burst Time and Priority for P4: 2 4 3

Process AT      BT      PR      CT      WT      TAT
P1      0       4       0       4       0       4
P2      1       2       1       6       3       5
P3      3       5       2      11       3       8
P4      2       4       3      15       9      13

Average Waiting Time = 3.75
Average Turnaround Time = 7.50
```

Result

Thus, the process scheduling algorithms — First Come First Serve (FCFS), Shortest Job First (SJF), Round Robin (RR), and Priority Scheduling — were successfully implemented.